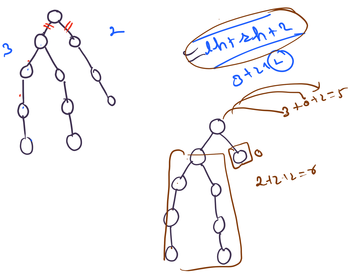
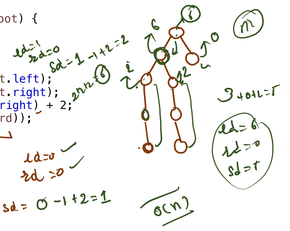
Lec-32

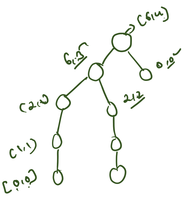
11/10/26 1:56 PM



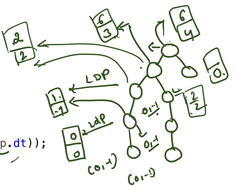
```
public int diameterOfBinaryTree(TreeNode root) {
        if (root == null) {
            return 0;
        }
        int ld = diameterOfBinaryTree(root.left);
        int rd = diameterOfBinaryTree(root.right);
        int sd = ht(root.left) + ht(root.right) + 2;
        return Math.max(sd, Math.max(ld, rd));
    }

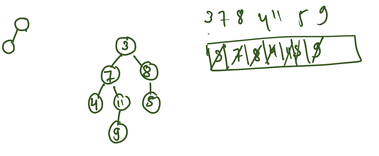
    private int ht(TreeNode node) {
        if (node == null) {
            return -1;
        }
        int lh = ht(node.left);
        int rh = ht(node.right);
        return Math.max(lh, rh) + 1;
    }
```

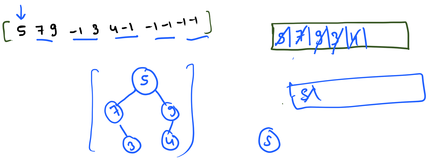




```
public DiaPair diameter(TreeNode root) {
        if (root == null) {
            return new DiaPair();
        }
        DiaPair ldp = diameter(root.left);
        DiaPair rdp = diameter(root.right);
        int sd = ldp.ht + rdp.ht + 2;
        DiaPair sdp = new DiaPair();
        sdp.dt = Math.max(sd, Math.max(ldp.dt, rdp.dt));
        sdp.ht = Math.max(ldp.ht, rdp.ht) + 1;
        return sdp;
    }
```







```
private void createTree() {
        // TODO Auto-generated method stubs
        Scanner sc = new Scanner(System.in);
        Queue<Node> q = new LinkedList<>();
        int x=sc.nextInt();
        root= new Node(x);
        q.add(root);
        while(!q.isEmpty()) {
            Node n  = q.poll();
            int c1=sc.nextInt();
            int c2=sc.nextInt();
            if(c1!=-1) {
                Node left=new Node(c1);
                n.left=left;
                q.add(left);
            }
            if(c2!=-1) {
                Node right=new Node(c2);
                n.right=right;
                q.add(right);
            }
        }

    }
```

